ESPE

UNIVERSIDAD DE LAS FUERZAS ARMADAS

INNOVACIÓN PARA LA EXCELENCIA

DEPARTAMENTO DE CIENCIA DE LA COMPUTACION

INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN (ITIN)

TEMA: AUTOEVALUACIÓN CAPITULO 2

INTEGRANTES:
LEOPOLDO BOLAÑOS
MEDINA MATIAS
MARTIN PALACIOS

ASIGNATURA: METOD DE DESARROLLO SOFTWARE

NRC: 30693

FECHA: 13/04/2026

DOCENTE: ING.JENNY ALEXANDRA RUIZ ROBALINO

FORMATO PARA ELABORACIÓN DE TALLERES ESTUDIANTILES	Departamento: **Ciencias de la Computación**
	Carrera: **ITIN** Asignatura: **Metodología de Desarrollo de Software** NRC: **30693**
	Apellidos y nombres del estudiante: **G7**

1. Antecedentes

1.1 Segun Somerville (2011), desde los inicios de la ingenieria de software, se ha identificado que el producto software no es solo el codigo fuente, sino que incluye programas, procedimientos, documentacion y datos relacionados. Esto es importante porque muchos estudiantes piensan que solo hay que escribir lineas de codigo y ya, pero la realidad es que el software como producto es mucho mas complejo y necesita de una estructura completa para que sea util y mantenible a largo plazo.

1.2 Pressman (2005) ya mencionaba que existen siete categorias de software, como software de sistemas, de aplicacion, incrustado y de inteligencia artificial, entre otras. Este autor destaca que cada tipo de software tiene caracteristicas propias y requiere enfoques diferentes para su desarrollo. En el contexto educativo, es fundamental que los futuros ingenieros conozcan estas categorias desde el principio, porque asi pueden elegir mejor las herramientas y metodologias segun el proyecto que van a realizar.

2. Objetivos

2.1. Identificar los componentes principales de un producto software, incluyendo los programas, la documentacion y los datos asociados, asi como comprender los atributos de calidad que debe cumplir segun el modelo ISO/IEC 25010, para que los estudiantes puedan aplicar estos conocimientos en el desarrollo de sus propios proyectos academicos (López Echeverry & Valencia Ayala, 2008; Somerville, 2011).

2.2 Diferenciar los tipos y clases de software existentes, como el software de sistemas, de aplicacion, incrustado y de ingenieria, con el fin de que los aprendices puedan seleccionar el lenguaje de programacion y las herramientas adecuadas segun el problema que necesiten resolver, empezando por lenguajes estructurados como C antes de pasar a otros mas complejos como Java o Python (IngSistemas, 2017; Pressman, 2005).

3. Desarrollo

3.1 Qué hay en la caja?

Dentro de la "caja" del producto software se encuentra un conjunto de programas, instrucciones, procedimientos y documentación asociada, ademas de los datos relacionados que permiten que el sistema funcione correctamente (Somerville, 2011). Osea, no solo es el código, sino todo lo que lo acompaña para que sea util.

3.2 ¿Cuál es el trabajo que tenemos que hacer para lograr la meta?

El trabajo pricipal para lograr la meta es seguir un proceso de desarrollo o ciclo de vida, el cual incluye actividades como el levantamiento de requicitos (funcionales y no funcionales), la elaboración del codigo fuente, las pruebas y finalmente la entrega del producto funcional (Somerville, 2011). Hay que hacer todo en orden para que salga bien.

3.3 ¿Qué tareas debemos realizar?

Las tareas que debemos realizar son varias: primero, entender que lenguaje de programación usar segun lo que se quiera hacer, como Java, C o Python; luego, escribir el codigo siguiendo buenas prácticas; también hay que probar constantemente para encontrar errores; y finalmente documentar todo el proceso (IngSistemas, 2017). Es mucho trabajo la verdad.

3.4 ¿Cómo nos aseguramos de que haremos un gran trabajo?

Nos aseguramos de hacer un gran trabajo aplicando los atributos de calidad del software, como la fiabilidad, la mantenibilidad y la facilidad de uso (Pressman, 2005). Tambien es importante seguir el modelo de calidad ISO/IEC 25010, que nos guia sobre las caracteristicas que debe cumplir el

producto para que sea de calidad (López Echeverry & Valencia Ayala, 2008). Sin calidad, el software no sirve de nada.

3.5 ¿Cómo medimos la calidad de lo que hemos hecho?

La calidad del producto software se mide mediante e l grado en que este satisface los requisitos especificados y las necesidades del cliente o usuario. Esto incluye revisar los atributos no funcionales como la seguridad, el rendimiento y la funcionalidad (Pressman, 2005). Usamos el modelo de calidad ISO 25010 para evaluar estas caracteristicas y subcaracteristicas.

3.6 ¿Cómo nos aseguramos de que lo que hemos hecho esté realmente a la altura de las necesidades y solicitudes iniciales?

Para asegurarnos de que el software cumple con lo que el cliente pidio desde el principio, debemos realizar pruebas de validación y verificación durante todo el proceso. Tambien es clave involucrar al cliente en las revisiones y hacer pruebas de aceptación al final (Somerville, 2011). Si no preguntamos al cliente, podemos equivocarnos feo.

3.7 ¿Cómo minimizamos el esfuerzo para lograr la meta?

Podemos minimizar el esfuerzo reutilizando componentes de software que ya existen y que han sido probados, como bibliotecas o módulos. Tambien usando herramientas de desarrollo integradas (IDE) y siguiendo estandares de codificación claros para no tener que rehacer el trabajo (IEEE Computer, 2014). Trabajar de forma inteligente, no solo duro.

3.8 ¿Qué habilidades necesitamos?

Necesitamos habilidades técnicas como saber programar en lenguajes como Java, C++ o Python, entender de algoritmos y estructuras de datos. Tambien se necesitan habilidades blandas como trabajar en equipo, comunicarse bien con el cliente y resolver problemas (IngSistemas, 2017). No todo es solo codigo, hay que hablar con la gente.

3.9 ¿Qué herramientas necesitamos?

Las herramientas necesarias incluyen editores de codigo, entornos de desarrollo integrados (IDES),

compiladores, sistemas de control de versiones como Git, y herramientas para hacer pruebas. Todo esto nos ayuda a construir el software de manera mas eficiente y ordenada (Somerville, 2011). Sin las herramientas correctas, es mucho mas dificil.

3.10 ¿Cuántas personas se necesitan?

La cantidad de personas depende del tamaño y la complejidad del proyecto. Para proyectos pequeños o aplicaciones de juguete, puede bastar con un solo desarrollador. Pero para sistemas complejos, se necesitan equipos grandes con diferentes roles: analistas, diseñadores, programadores y testers (Pressman, 2005). No es lo mismo hacer una app simple que un sistema grande.

3.11 ¿Qué habilidades, perfiles, experiencia?

Se necesitan perfiles variados: analistas de requisitos que entiendan lo que el cliente necesita, diseñadores que creen la arquitectura, programadores que escriban el codigo, y testers que busquen errores. La experiencia recomendable es empezar aprendiendo C estructurado para luego pasar a lenguajes más altos como Java o Python (IngSistemas, 2017). Cada quien tiene su rol y entre todos hacen el equipo.

4. Conclusiones

4.1 En resumen, el producto software es mucho mas que solo lineas de codigo, ya que incluye programas, procedimientos, documentación y datos relacionados, tal como lo señala Somerville (2011). Por lo tanto, un estudiante o futuro ingeniero no puede enfocarse unicamente en programar, sino que debe entender que el software como producto requiere de un proceso completo que incluye desde el levantamiento de requisitos hasta las pruebas y la entrega final. Si no se toman en cuenta todos estos elementos, el software probablemente va a tener problemas de calidad, va a ser dificil de mantener y no va a cumplir con lo que el cliente realmente necesita. Ademas, los atributos de calidad como la fiabilidad, la seguridad y la mantenibilidad son fundamentales para que el producto sea exitoso (Pressman, 2005).

4.2 Otra conclusion importante es que existen diferentes tipos de software, como el de sistemas, el de aplicacion, el incrustado y el de inteligencia artificial, y cada uno tiene caracteristicas y enfoques

distintos (Pressman, 2005). Esto significa que no se puede usar la misma receta para todos los proyectos. Por ejemplo, desarrollar una aplicacion para un telefono Android no es lo mismo que hacer un sistema de control para una maquina industrial. Por eso, los estudiantes deben aprender primero conceptos solidos con lenguajes estructurados como C, y luego avanzar a lenguajes mas complejos como Java o Python (IngSistemas, 2017). Si se salta esta base, despues le va a costar mucho entender cómo funcionan las cosas y va a cometer errores que podrian haberse evitado.

Recomendaciones

Recomendacion 1: Se recomienda que los estudiantes que estan iniciando en el desarrollo de software aprendan primero un lenguaje estructurado como el lenguaje C, porque esto les facilitará despues aprender otros lenguajes de mas alto nivel como Java o Python. Como dice IngSistemas (2017), quien conoce bien C va a tener menos dificultades para interpretar y escribir codigo en otros lenguajes. No hay que saltarse pasos, porque sino despues cuesta mucho entender conceptos basicos.

Recomendacion 2: Es muy importante que los equipos de desarrollo no solo se enfoquen en que el software funcione, sino que tambien midan la calidad del producto usando los atributos del modelo ISO/IEC 25010, como la fiabilidad, la seguridad y la mantenibilidad. Segun Pressman (2005), un buen software no es solo el que hace lo que el cliente pide, sino el que ademas es facil de mantener y usar. Por eso, desde el principio hay que pensar en la calidad, no solo en terminar rapido.

Referencias

IEEE Computer, S. (2014). Guide to the Software Engineering Body of Knowledge Version 3.0 (SWEBOK Guide V3.0). https://www.academia.edu/8583868/Guide_to_the_Software_Engineering_Body_of_Knowledge_Version_3_0_SWEBOK_Guide_V3_0

IngSistemas. (2017). ¿Qué es la ingeniería de software? https://ing-sistemas.com/2017/02/09/que-es-la-ingenieria-software/

López Echeverry, A. M., & Valencia Ayala, L. E. (2008). Introducción a la calidad de software. Dialnet. https://dialnet.unirioja.es/servlet/articulo?codigo=4745899

Pressman, R. (2005). Software Engineering: A Practitioner's Approach (7th ed.). McGraw-Hill.

Somerville, I. (2011). Software Engineering (9th ed.). https://ifs.host.cs.st-andrews.ac.uk/Books/SE9/index.html

Anexos